

Realm Shift Runner

Gameplay Programming Assessment

Author: Abdulmajeed Alshammari

1. System Overview

Inspired by the core gameplay loop of mobile endless runners, the project introduces Realm Shift, a seamless world-switching mechanic that combines instant visual transformation with meaningful gameplay decisions. Players switch between Fantasy and Sci-Fi realms within the same scene, creating a fast-paced experience where visual feedback and gameplay decisions work together as a single mechanic.

2. Gameplay Twist & Innovation

Rather than replacing the foundations of the endless runner genre, the system preserves its familiar gameplay loop—including auto-run, three-lane movement, jumping, sliding, progressive difficulty, and endless pattern generation—while introducing a new strategic layer through Realm Shift.

Realm Shift allows the player to instantly switch between two parallel worlds without scene loading or gameplay interruption. More importantly, it is not merely a visual transition. The mechanic is deeply integrated into the gameplay loop, requiring players to switch realms at the correct moment to bypass dead ends and survive. A missed or delayed realm shift results in failure, making it a core gameplay decision rather than a cosmetic effect.

This transforms the traditional endless runner from a purely reaction-based experience into one that also rewards anticipation, timing, and strategic decision making, adding a new dimension to the genre while preserving its fast-paced nature.

From an engineering perspective, the mechanic demonstrates how gameplay logic can remain independent from presentation, enabling seamless world transitions through a modular and reusable architecture.

3. Core Gameplay System

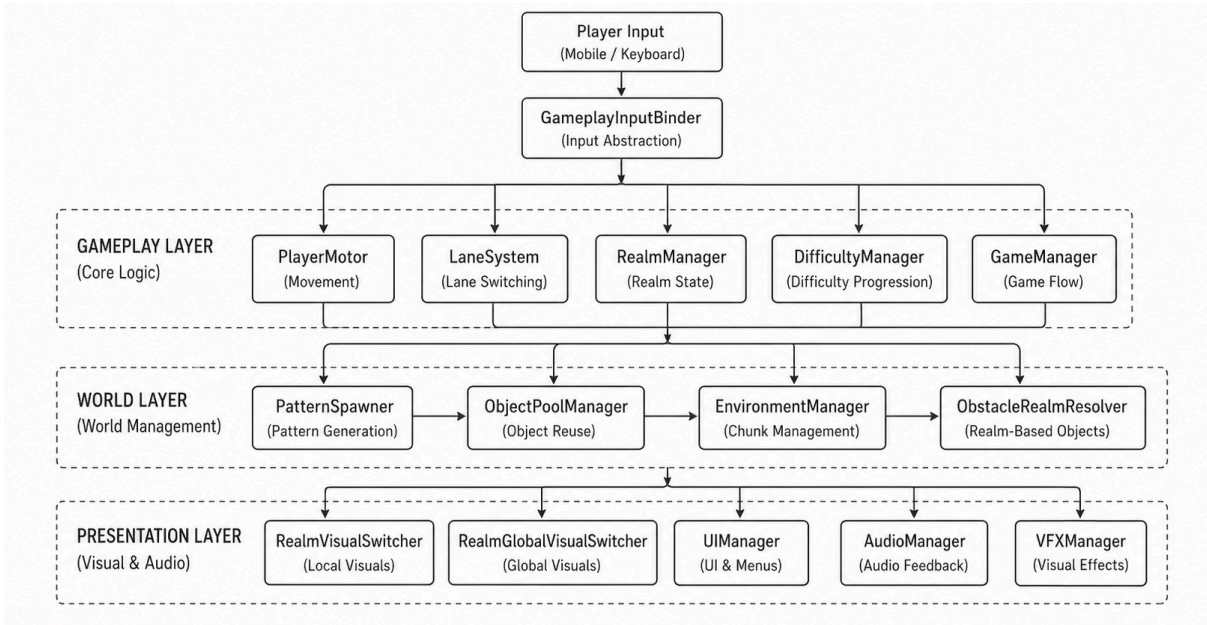
The gameplay system includes:

- Instant Realm Shift (the core mechanic)
- Auto Run
- Three-Lane Movement
- Jump
- Slide
- Pattern-Based Spawning
- Progressive Difficulty Scaling

Player feedback is provided through character animations, realm transition visuals, audio, VFX, and UI.

These systems work together to create a responsive and polished gameplay experience while maintaining a limited and focused project scope.

4. Architecture: The project follows a modular, data-driven architecture where gameplay logic is separated from presentation systems. Gameplay configuration is externalized using ScriptableObjects, allowing systems to remain flexible, reusable, and easy to extend.



Key Scripts	Responsibility
GameplayInputBinder	Input abstraction for mobile and keyboard
PlayerMotor	Player movement and actions
LaneSystem	Three-lane movement
RealmManager	Active realm state
DifficultyManager	Controls gameplay progression and difficulty scaling
OpticalPatternSpawner	Endless pattern generation
ObjectPoolManager	Runtime object reuse
RealmVisualSwitcher	Local realm presentation
RealmGlobalVisualSwitcher	Global visual changes

5. Key Engineering Decisions:

- Single-Scene Realm Switching to eliminate gameplay interruption.
- Gameplay / Presentation Separation for modularity.
- Data-Driven Design using ScriptableObjects for flexible gameplay configuration.
- Object Pooling to avoid unnecessary runtime allocations.
- Pattern-Based World Generation for reusable level content.

6. Repository

The complete Unity project, source code, and setup instructions are available in the accompanying Git repository: